

CAHIER DES CHARGES : MIDDLEWARE & SOBRIÉTÉ IA MESURÉE

Sujet du Sprint : proxy d'éco-conception IA et routage frugal

Piliers RSE : Environnemental · Analyse du cycle de vie, performance énergétique & usage raisonné de l'IA

1. CONTEXTE & ENJEUX TECHNIQUES

À l'échelle individuelle, l'envoi d'un prompt isolé semble immatériel. Pourtant, lors d'un déploiement industriel ou d'une utilisation répétée en entreprise, l'absence de stratégie de mise en cache, la longueur non contrôlée des réponses et le choix systématique de modèles géants font exploser l'empreinte carbone, la consommation d'eau des datacenters et les coûts financiers.

La sobriété architecturale en IA consiste à instrumenter, mesurer et intercepter les flux pour appliquer une stratégie de calcul raisonnée.

2. MISSION DE L'ÉQUIPE

Développer une API Backend (Node.js ou Python) faisant office de **Proxy d'éco-conception IA**. Ce middleware interceptera les requêtes destinées aux LLM pour optimiser la consommation de ressources.

Pour valider votre travail, vous devrez simuler un trafic de 100 requêtes et prouver scientifiquement le gain environnemental cumulé entre une gestion brute et votre middleware responsable.

3. PÉRIMÈTRE FONCTIONNEL & TECHNIQUE (LE MVP)

L'application doit obligatoirement intégrer :

- **Un serveur backend fonctionnel** exposé localement (Express/Fastify pour Node, FastAPI/Flask pour Python).
- **Un simulateur de charge** capable de rejouer un jeu de test de 100 prompts (comprenant des requêtes répétitives, des questions simples et des tâches complexes).
- **Une intégration de l'outillage de mesure** (SDK EcoLogits ou un modèle mathématique basé sur ses données) pour calculer l'impact en temps réel.

4. SPÉCIFICATIONS DES SPRINTS

Sprint 1 : Le Mode naïf & Instrumentation

1. **Mise en place de l'API** : Créez un endpoint de proxying standard qui reçoit un prompt et le transfère directement à une API de LLM lourd (réelle ou simulée, type GPT-4).
2. **Configuration naïve** : Le proxy n'applique aucun filtre : les prompts passent tels quels, le nombre de tokens retournés n'est pas limité, et aucune mise en cache n'est active.
3. **Audit "Avant"** : Lancez votre script de 100 requêtes à travers ce proxy naïf. Utilisez la librairie *EcoLogits* pour enregistrer et compiler l'impact environnemental total.
 - *Livrable intermédiaire* : Un rapport d'impact initial brut (Wh cumulés, gCO2 e, et eau en ml).

Sprint 2 : Le Mode Responsable & Logique Frugale

Refactorisez votre middleware pour y intégrer obligatoirement **trois couches d'éco-conception logicielle** :

- **Couche 1 : Cache sémantique ou exact (Exact/Semantic Caching)** : Si un prompt identique (ou sémantiquement très proche) a déjà été traité récemment, le middleware doit intercepter la requête et renvoyer immédiatement la réponse stockée en mémoire (ou via un cache local léger type Redis), court-circuitant totalement l'appel à l'IA.
- **Couche 2 : Routage dynamique frugal (Dynamic Routing)** : Analysez la complexité du prompt entrant. Si la tâche est simple (ex: une classification de texte, une correction, ou une demande de structure simple), routez automatiquement la requête vers un modèle beaucoup plus léger et frugal (ex: Mistral-7B local ou émulé). Ne réservez le gros modèle qu'aux requêtes hautement complexes.
- **Couche 3 : Troncature et contrôle du payload** : Forcez une limite stricte sur le nombre de tokens générés en sortie (*max_tokens*) pour éviter les réponses inutilement verbeuses et énergivores.
- **Injection de Headers** : Votre middleware doit injecter dynamiquement l'impact environnemental calculé de la requête dans les en-têtes de la réponse HTTP renvoyée au client (ex: X-Calculated-Impact-Wh, X-Calculated-Impact-gCO2e).

5. BUDGET D'IMPACT & OBJECTIFS À ATTEINDRE (KPIs)

Pour valider la réussite du sprint, votre V2 (Mode Responsable) exécutée sur le même jeu de 100 requêtes doit prouver les performances suivantes :

- **Taux de hit du cache** : Minimum 25% de requêtes interceptées et résolues localement (grâce aux prompts répétés dans le jeu de test).
- **Réduction de l'empreinte carbone et électrique** : Une diminution mathématique d'**au moins 60%** de la consommation globale d'énergie (Wh) et des gaz à effet de serre (gCO2 e) par rapport à la V1.

6. STACK TECHNIQUE SUGGÉRÉE

- **Backend** : Node.js (JavaScript/TypeScript) ou Python.
- **Librairie d'impact** : SDK *ecologits* ou utilisation des données de l'API *Compar:IA* pour simuler les facteurs d'émissions par modèle.
- **Cache** : Stockage en mémoire vive (Object/Map natif) ou instance locale Redis.

7. LIVRABLES

Lors de votre démo live de 5 minutes, vous devrez présenter au

- **Le Dashboard / Tableau de bord GreenOps** : Une synthèse claire affichant les métriques cumulées Avant vs Après (Wh, gCO2 e, Eau) et le pourcentage d'économie réalisé.
- **La démonstration des Headers** : Faire une requête en direct (via un client HTTP ou l'inspecteur) et montrer que les en-têtes HTTP contiennent bien les données d'impact calculées par votre code.
- **L'efficacité du Cache** : Prouver visuellement qu'un prompt répété renvoie une réponse instantanée à coût carbone égal à 0.
- **La revue de code** : Justifier l'implémentation de vos algorithmes de routage et de filtrage des prompts.